4 - Recursion

In programming we are typically operating on sets. In order to support more effective coding we typically rely on structures that allow us to repeat a number of operations on each element of a set so that we can write compact and easily adaptable code. This follows from the reasoning that if we are operating on a set, we are likely performing the same operations on each member of the set, so the code is highly redundant and might be modeled in a way that allows a small sequence of code to be repeated however many times there are elements in the set. In an introductory programming class, we introduce the `for` loop control element for this purpose and we reuse it extensively; however, this is not the only means of repeating code. In this class, we will generally refer to repeating a set of code as **iteration** and the design model of using a `for` loop as the basis of repeating in an algorithm as an **iterative** design, technique, or approach.

An alternative approach for the design of an algorithm is to repeat code by calling a function repeatedly. This requires close attention to the design of the function as a poor design with an unreachable return may result in the function calling itself until the system runs out of memory which would result in a program crash at best and a system crash at worse. In this class, we will exclusively refer to repeating a set of code using a function as **recursion** and the design model of using a function as the basis of repeating in an algorithm as a **recursive** design, technique, or approach.

Let's discuss recursion in terms of some basic mathematical functions such as power, factorial, and the Fibonacci sequence.

Power

Power is a common alternative name for an exponential operation. For the sake of this discussion, assume power is defined for zero and positive exponents only. In other words, the power function defined in this section is discrete and computes exponentials for only non-negative integer exponents and non-zero bases.

Define power as $f(a, b)$ where a is the base and b is the exponent. Power is often expressed using the notation $f(a, b) = a^b$; however, this notation masks that the power operation is really defined as a repeated multiplication per Definitions 1 and 2.

Definition 1 :

$$f(a, b) = 1 \times a_1 \times \cdots \times a_b$$

Definition 2 *(As a piecewise formula)* :

$$f(a, b) = \begin{cases} 1 & b = 0 \\ a \cdot f(a, b-1) & b > 0 \end{cases}$$

We may use different symbols to make the operation more descriptive, *e.g.* `power(base, exp)`, rather than using the symbolic representation of math, but doing so does not change the definition. Regardless of how the power operation is represented, it is a function with two parameters where the base is typically the first parameter and the exponent is typically the second parameter. For simplicity, we are defining power as discrete, *i.e.* non-negative, integer exponents and integer, non-zero bases. While limited, it is an incredibly useful version of power because we often need exponentials in the integer domain.

Power Iteratively

Let's implement using an iterative approach, *i.e.* using Definition 1. What does an implementation look like?

```
power(b, e) → integer >= 1 where b and e are integers and b is >= 1 and e >= 0 :

    result = 1

    for i = 1 to e :
        result = result * b

    return result
```

Consider the efficiency of the iterative power implementation. The number of operations is dependent on the number of times the loop iterates. The number of loop iterations is determined by the exponent input `e`. The mathematical relationship between the number of loop iterations and `e` is linear. In other words, if `e` increases by one, the number of overall operations increases according to a linear relationship. `b` is a constant in this analysis and can be generally dismissed with respect to its affect on time complexity. Therefore the overall time complexity is $O(n)$ or linear time complexity.

Power Recursively

By Definition 2, the piecewise mathematical expression defines a clear model for how to implement this recursively:

$$f(a,b) = \begin{cases} 1 & b = 0 \\ a \cdot f(a, b-1) & b > 0 \end{cases}$$

The two independent pieces or cases of the piecewise formula need to be implemented. The first case represents a fundamental basis of the problem. In the case of power, this is the axiom that for all natural number bases, a zero exponent produces a result of 1, i.e. $a^0 = 1$ for all a in the domain of natural numbers. This may also be called a "base case".

```
power(b, e) → integer >= 1 where b and e are integers and b is >= 1 and e >= 0 :
    if( e == 0 ) :
        return 1

    return b * power(b, e - 1);
```

Consider the efficiency of the recursive power implementation. Every time that `power` is called, `power` subtracts 1 from `e`. Assuming `e` is positive (a fundamental design assumption), `e` takes one step closer to the base case. The total number of calls to `power` is determined by the initial value of the exponent e_i on the first external call to `power` which will result in i internal calls to `power`. This means there is a linear relationship between the input `e` and the total number of calls to `power`. Again, the base `b` is effectively constant and does not affect the time complexity, so it is ignored. The linear relationship between the size of input `e` and the number time `power` calls itself means that this implementation is $O(n)$ and has linear time complexity.

Recursive Design

All recursive functions must include two control cases

1. Base Case
2. Recursive Case

The base case represents an immutable fundamental solution to one of the smallest versions of this problem.

The recursive case represents reducing the problem into constituent problems, part of which is a self-similar subproblem, *i.e.* it has different, smaller parameters and is closer to a base case but it is fundamentally structured as the same type of problem. We generally control a recursive algorithm by parameter reduction when making a recursive function call which leads subsequent recursive function calls towards a base case.

By viewing recursive problems as composed of self-similar subproblems, we can properly structure our algorithms in a reductive manner. In other words, we typically want our design to count down towards base cases designed around when one or more parameters approach or reach zero. Good recursive designs generally approach zero or one because it is hard to miss and clearly associates base cases with the smallest indivisible subproblem. When a recursive method is structured to count up to a target number, it requires additional parameters and every parameter increases the memory usage of the recursive function.

Recall that all local variables use stack memory and all parameters are local variables, so adding an additional parameter increases the amount of memory required to support this function. There is a finite amount of memory in a computer, so increasing the number of local variables decreases the number of times a recursive function can be called compared to a function with less local variables. Since memory is finite, the stack, which provides memory for local variables, will eventually run out of space if functions are repeatedly called without returning. Running out of memory is typically called a Stack Overflow exception and leads to a crash of the program. In modern systems, there is so much stack memory, it is difficult to run out of memory unless your recursive method is improperly structured and cannot return properly or if the input size is exceptionally large.

Factorial

Assume factorial is defined for positive integers only. In other words, factorial is a discrete function for positive inputs only. The definition involves repeated multiplication as indicated in Definitions 3 and 4.

Definition 3 :

$$f(x) = 1 \times 2 \times \cdots \times (x-1) \times x$$

Definition 4 *(As a piecewise formula)* :

$$f(x) = \begin{cases} 1 & x = 1 \\ x \cdot f(x-1) & x > 1 \end{cases}$$

####

Factorial is often expressed symbolically using the notation $f(x) = x!$; however, this notation is problematic in programming languages as it is odd and unique syntax for an operator to follow an operand in most programming languages. Instead factorial is generally only accessible via a function with a typical declaration being `int factorial( int x )`.

Factorial Iteratively

The iterative implementation of factorial follows the model prescribed by Definition 3.

```
factorial(x) → integer > 0 where x is an integer and x is > 0 :
    fx = 1

    for i = 2 to x
        fx = fx · i

    return fx
```

Just as with the `power` iterative implementation, the number of operations for the `factorial` function scales with the loop and the number of operations is dependent upon the input `x`. The relationship between the number of loop iterations and the input `x` is linear which means that this iterative factorial implementation is $O(n)$ or linear time complexity.

Factorial Recursively

By Definition 4, the piecewise model for factorial suggests the recursive implementation of `factorial` the same way Definition 2 suggests the recursive implementation of `power`.

$$f(x) = \begin{cases} 1 & x = 1 \\ x \cdot f(x-1) & x > 1 \end{cases}$$

Again, factorial has a single base case defining the basis of the factorial definition and the ultimate exit condition for the operation. Because the piecewise mathematical definition of factorial is structurally similar to the piecewise mathematical definition of power, the implementation of both is almost identical with the exception of literals and operators.

```
factorial(x) → integer > 0 where x is an integer and x is > 0 :
    if x = 1 :
        return 1

    return x · factorial(x-1)
```

For the same reasons we conclude that the efficiency of recursive `power` is $O(n)$, we generally estimate the `factorial` to have linear time complexity. Just as in `power`, every time that `factorial` is called, `factorial` subtracts 1 from `x` and `x` takes one step closer to converging on the base case.

Fibonacci Sequence

The Fibonacci sequence is defined by the series 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

In other words the i-th Fibonacci Number is the number in the i-th position of the Fibonacci Sequence. By definition, the Fibonacci Sequence is only defined for non-negative integers, begins with $f(0) = 0$ and $f(1) = 1$, and subsequent Fibonacci Numbers are defined as the sum of the two preceding Fibonacci Numbers. Here is the Fibonacci sequence with corresponding indices:

x	0	1	2	3	4	5	6	7	8	9	10	11	12	...
f(x)	0	1	1	2	3	5	8	13	21	34	55	89	144	...

Definition 5 *(As a piecewise formula)* :

$$f(x) = \begin{cases} 0 & x = 0 \\ 1 & x = 1 \\ f(x-1) + f(x-2) & x > 1 \end{cases}$$

Fibonacci Iteratively

While we do not have a simple, clean mathematical formula representing the iterative form for computing a Fibonacci Number, we can implement it using a loop:

```
fibonacci(x) → integer >= 0 where x is an integer and x is >= 0 :
    fx-2 = 0
    fx-1 = 1
    fx = fx-2

    for i = 1 to x:
        fx = fx-1 + fx-2
        fx-2 = fx-1
        fx-1 = fx

    return fx
```

Comparing the above implementation with iterative power or factorial, iterative Fibonacci involves at least 5 variables and appears to generally be a more complex implementation; but, how does this affect its time complexity. If we were trying to compute an accurate measure of efficiency, we might quantify the above implementation as $4n + 4$. The constant term captures the first three statements and the `return` statement while the coefficient captures the top of the `for` loop and all of the statements in the loop itself. Keep in mind that this is an estimate in itself as there are aspects of this code that will be hidden by any compiler hence why we use an estimation methodology that is elastic and does not pretend to be too accurate. If we ignore coefficients and constants, we conclude that the efficiency of the implementation is linear time complexity and has a Big O of $O(n)$.

Fibonacci Recursively

Definition 5 prescribes the recursive model for Fibonacci:

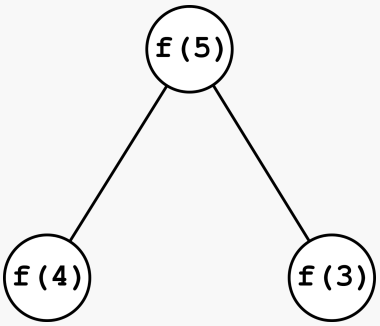
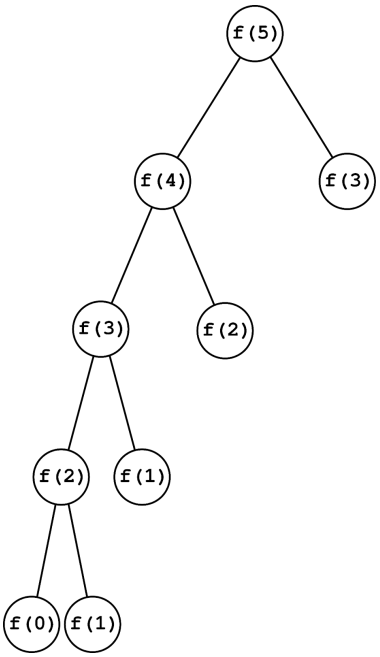
$$f(x) = \begin{cases} 0 & x = 0 \\ 1 & x = 1 \\ f(x-1) + f(x-2) & x > 1 \end{cases}$$

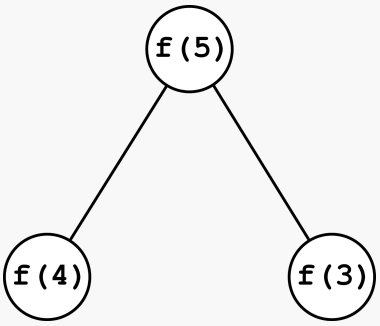
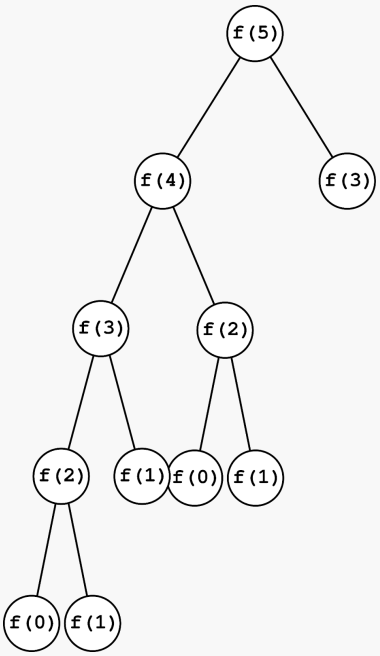
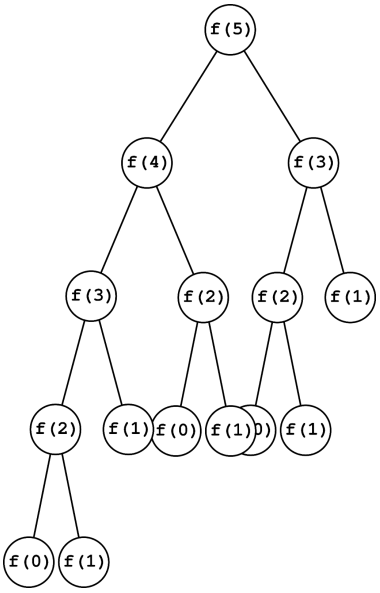
This definition is so far unique as it prescribes two base cases and a more complex recursive case when compared with Definition 2 for power or Definition 4 for factorial; however, the implementation is still rather simple.

```
fibonacci(x) → integer >= 0 where x is an integer and x is >= 0 :
    if x = 0 :
        return 0
    if x = 1 :
        return 1

    return fibonacci(x-1) + fibonacci(x-2)
```

Again, consider the efficiency of Fibonacci. While most of the design elements are familiar, a novel question about how to estimate the efficiency of the recursive case needs to be considered.

If the recursive case is reached in Fibonacci, how many function calls result? The answer is two. This means for each call to recursive Fibonacci, the algorithm potentially generates two additional recursive calls. A diagram may help illustrate the grown of function calls in recursive Fibonacci:

The initial call to recursive Fibonacci with say an input of 5, will require that it solves two smaller Fibonacci problems, *i.e.* and . Given this pattern, for each call we expect two additional calls.	
Recursive Fibonacci will actually start to solve itself by repeatedly calling down until it has solved the left most $f(2)$ subproblem in an effort to solve the left most $f(3)$ subproblem in an effort to solve the left hand $f(4)$ subproblem. Once it has solved the left hand $f(2)$ subproblem, it can very easily solve the left hand $f(3)$ subproblem because the solution involves the just solved $f(2)$ solution and a base case $f(1)$ solution. After the left most $f(3)$ subproblem is solved, Fibonacci will solve the left most $f(4)$ subproblem.	

The initial call to recursive Fibonacci with say an input of 5, will require that it solves two smaller Fibonacci problems, *i.e.* $f(4)$ and $f(3)$. Given this pattern, for each call we expect two additional calls.	
To complete computing $f(4)$ after solving the $f(3)$ calculation, *i.e.* the left most subtree, recursive Fibonacci then needs to solve the $f(2)$ problem again.	
After solving the entire $f(4)$ subproblem, *i.e.* the entire left subtree, recursive Fibonacci must solve the entire $f(3)$ subproblem, *i.e.* the entire right subtree, again.	

Early in the course, we discovered that the number of nodes in a structure like this is at most 2^n where n is the number of nodes or the size of the input under certain conditions that we will discuss later when we focus specifically on trees. It does not matter that some of the subtrees are incomplete. Imagine the growth of the tree, *i.e.* the number of additional nodes that would be appended, if we added one more level above to solve $f(6)$. The

size of the tree would effectively double. In other words, recursive Fibonacci has exponential time complexity or it is classified as $O(2^n)$. This time complexity is also evident from the perspective offered by the impllementation

```
fibonacci(x) → integer >= 0 where x is an integer and x is >= 0 :
    if x = 0 :
        return 0
    if x = 1 :
        return 1

    return fibonacci(x-1) + fibonacci(x-2)
```

Note that one call to the function generally produces two calls, so the initial call to Fibonacci results in two more calls. However, these two calls in turn produce four calls which produce eight calls and so forth. So what is the relationship between the size of the input n (`x` in the pseudocode) and the number of calls. Consider the sequence of calls described :

$$1 + 2 + 4 + 8 + \ldots$$

What is the formula that describes the number of calls:

$$2^0 + 2^1 + 2^2 + 2^3 + \ldots$$

While the formula does not precisely model the number of calls, it is a reasonable approximation for the number of calls. If we assume a full tree is generated, the formula becomes:

$$2^0 + 2^1 + 2^2 + 2^3 + \ldots + 2^{n-1} + 2^n$$

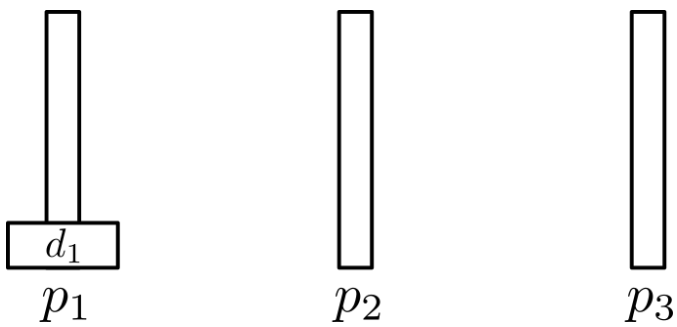
This sum is equal to:

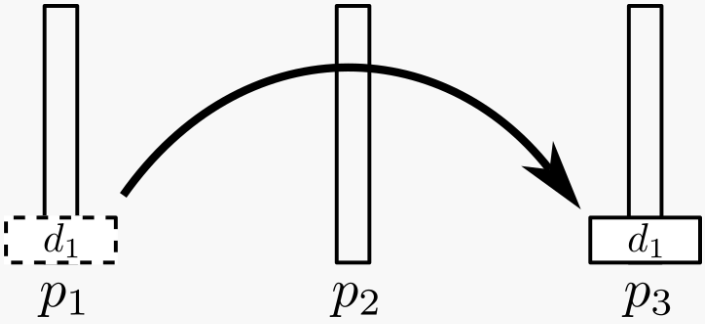
$$2^{n+1} - 1$$

Since we are performing an efficiency analysis, we can drop constants in the equation and summarize this as $O(2^n)$ or exponential time complexity.

Towers of Hanoi

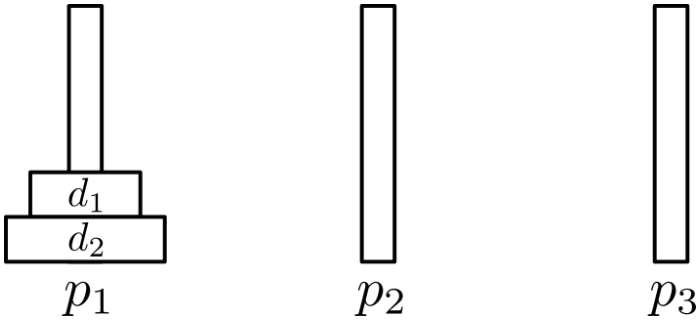
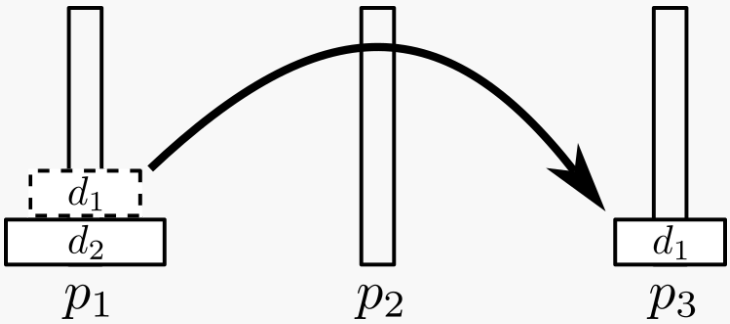
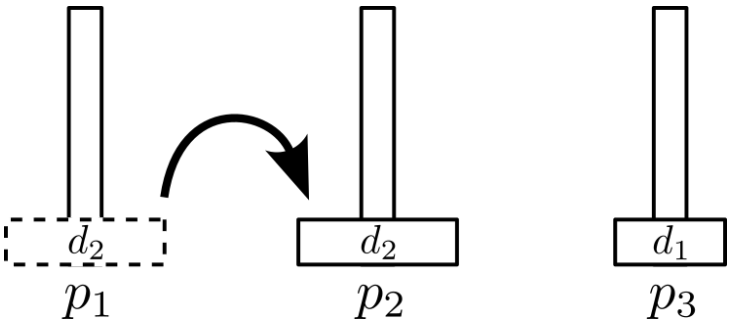
Towers of Hanoi is based on a legend involving Buddhist monks moving large, circular stone discs from one peg to another. The problem is to move a stack of discs from one peg to another by moving only a single disc at a time.

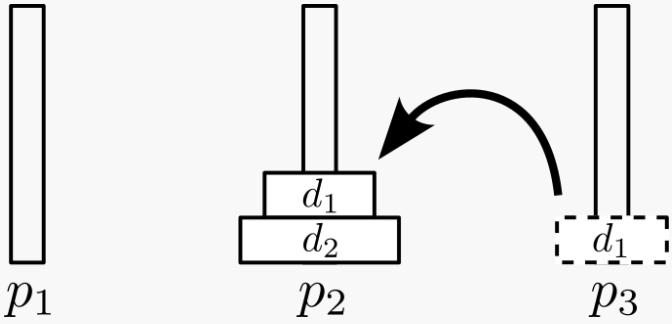
	One Disc Towers of Hanoi
initial	

	One Disc Towers of Hanoi
move 1	

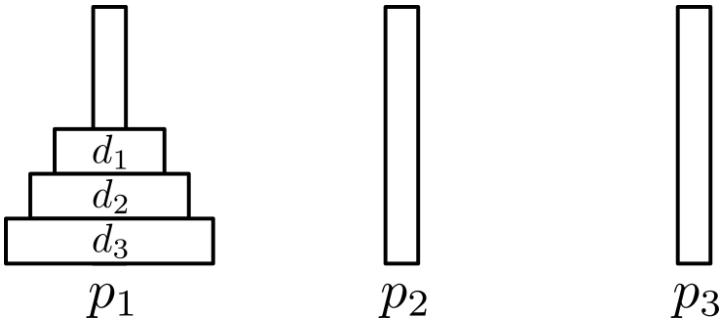
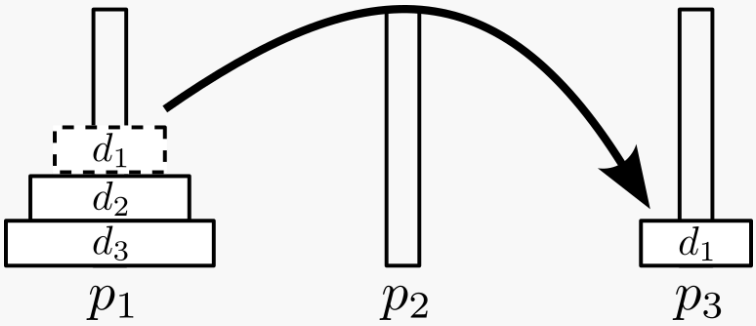
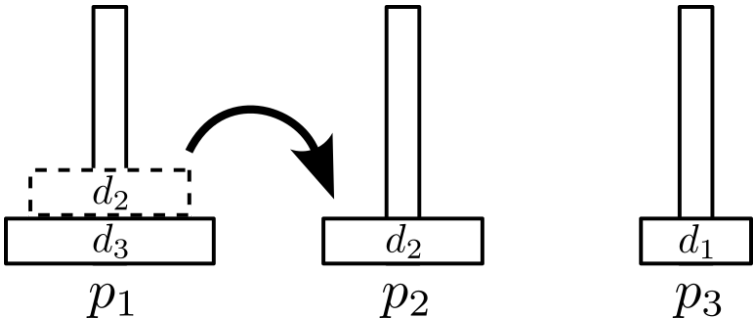
The single disc problem is trivial and requires only a single move to complete the puzzle.

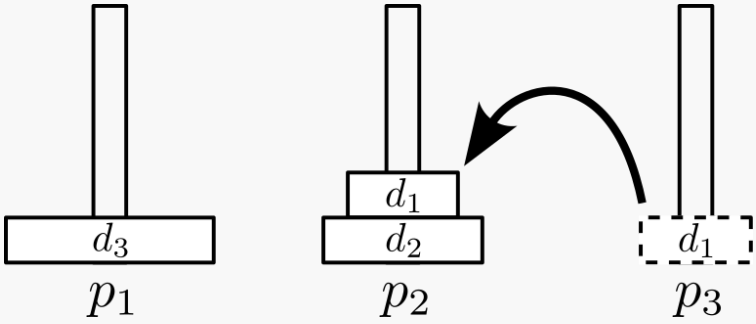
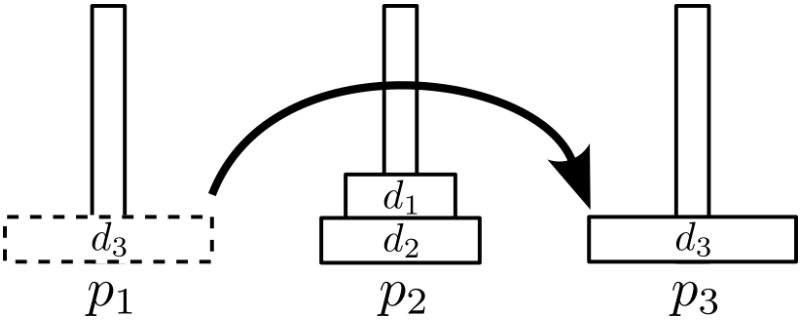
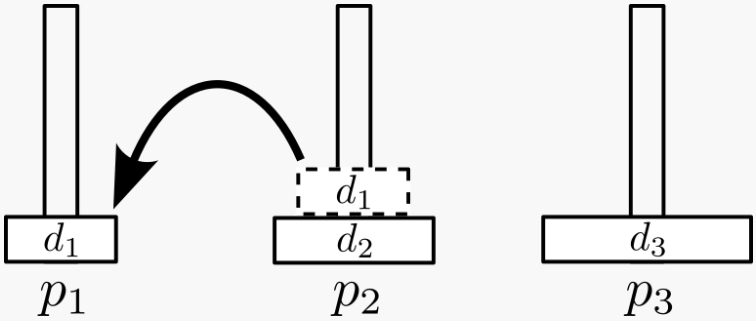
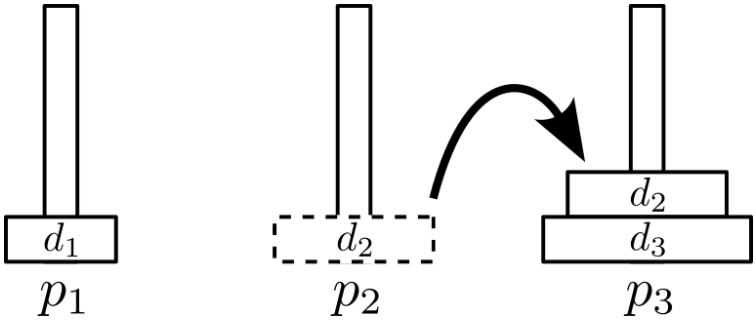
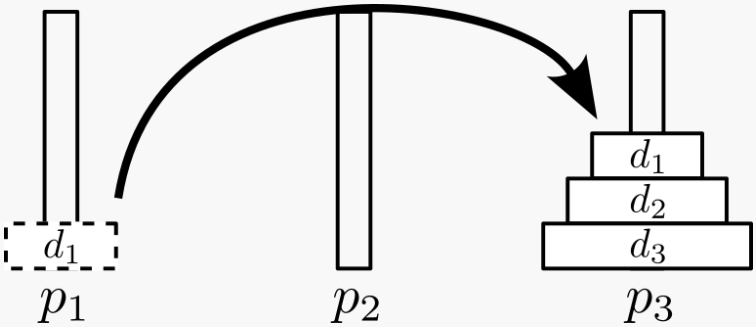
Each disc must have a different diameter than all other discs, and for a multiple disc problem, no disc that is larger than another disc can be stacked on top of a smaller disc by rule. Therefore, to move the two discs to another post and restack them in the original order, it takes three moves.

	Two Disc Towers of Hanoi
initial	
move 1	
move 2	

	Two Disc Towers of Hanoi
move 3	

The three disc tower increases the complexity of movement.

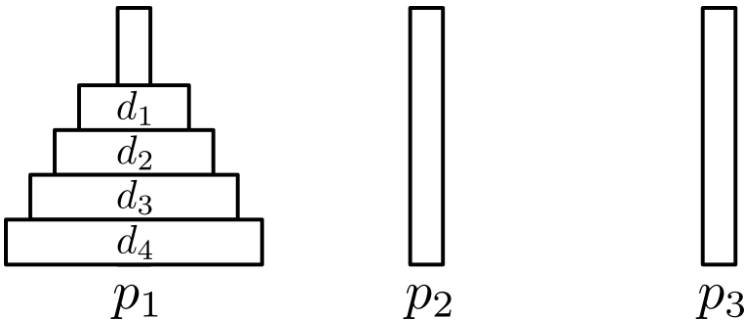
	Three Disc Towers of Hanoi
initial	
move 1	
move 2	

	Three Disc Towers of Hanoi
move 3	
move 4	
move 5	
move 6	
move 7	

However, a key insight is that the three-disc Towers of Hanoi is unstacking a two-disc tower, moving one disc, restacking a two-disc tower. In other words, the solution for the three-disc tower requires solving a two-disc tower twice which suggests that we can approach this problem using recursion.

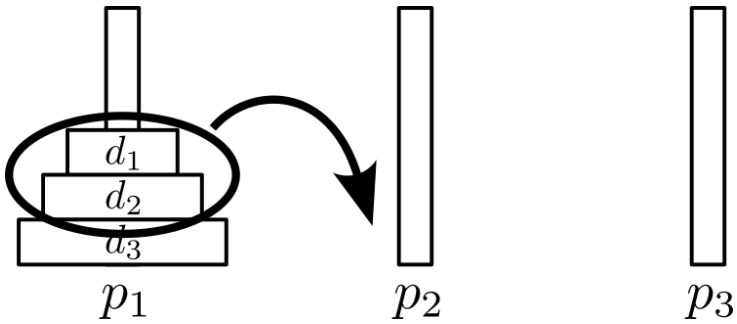
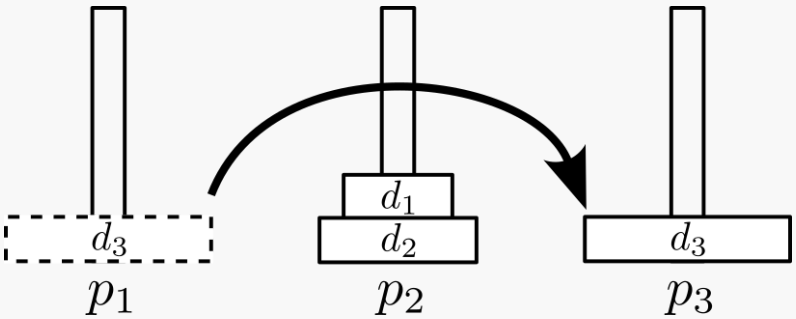
Algorithmic Time Complexity

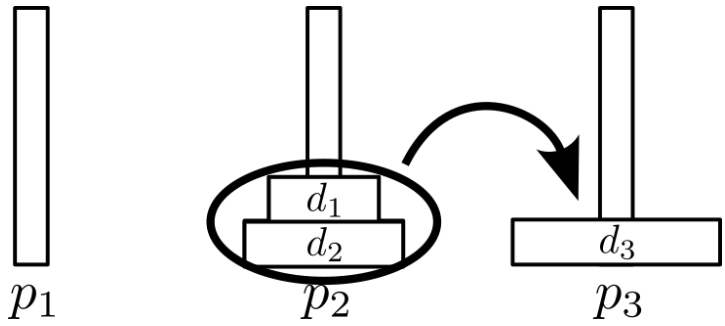
How many moves are necessary to solve Towers of Hanoi for four discs or the general n-disc problem?



Each single action, a discrete move, is always the same, *i.e.* move a single disc. For a stack of discs, we are just repeating the move single disc action a number of times and to find the time complexity it is just a matter of estimating how many move disc actions are necessary to solve for a tower of n-discs where the number of discs, n, is the "size of the input".

Further, consider that the three disc problem must solve the two disc problem twice. Once the two discs above the third disc are stacked elsewhere, moving the bottom-most disc of the current problem is just a single move, but then the two disc problem must be solved again to replace the discs above on top of the bottom-most disc.

solve 2 disc tower to free d_3	
move d_3	

solve 2 disc tower to complete solution	

Therefore the solution to the current problem of n-discs is just a 1 disc problem in between solving the n-1 disc problem twice. The n-1 disc problem is just solving a 1 disc problem in between solving the n-2 disc problem twice, and so forth. Based upon our understanding of recursion, this is clearly a problem that can be decomposed into self-similar subproblems with a base case of moving a single disc.

To estimate time complexity, we can refer back to what we learned from recursive Fibonacci where a call to Fibonacci resulted in two additional calls to Fibonacci which produced a tree that we agreed had approximately an exponential number of elements with respect to the size of the input. We see this same pattern here in Towers of Hanoi as every time we add a disc, it results in having to solve two smaller $n-1$ Towers of Hanoi problems which in turn results in solving four smaller $n-2$ Towers of Hanoi problems and so forth. We can summarize this with the piecewise recursive equation:

$$f(d) = \begin{cases} 1 & d = 1 \\ f(d-1) + 1 + f(d-1) & d > 1 \end{cases}$$

We can also determine the total of moves from an inductive approach. Given the following pattern:

1 disc $\rightarrow$ 1 move

2 disc $\rightarrow$ 1+1+1 = 3 moves

3 disc $\rightarrow$ 3 + 1 + 3 = 7 moves

Based on the progression, the 4-th element will be:

4 disc $\rightarrow$ 7 + 1 + 7 moves

But what is the i-th?

The insight comes from understanding that the right side of the equation is related to the left by powers of 2. In other words:

i disc = $2^{i-1} - 1 + 1 + 2^{i-1} - 1$ moves

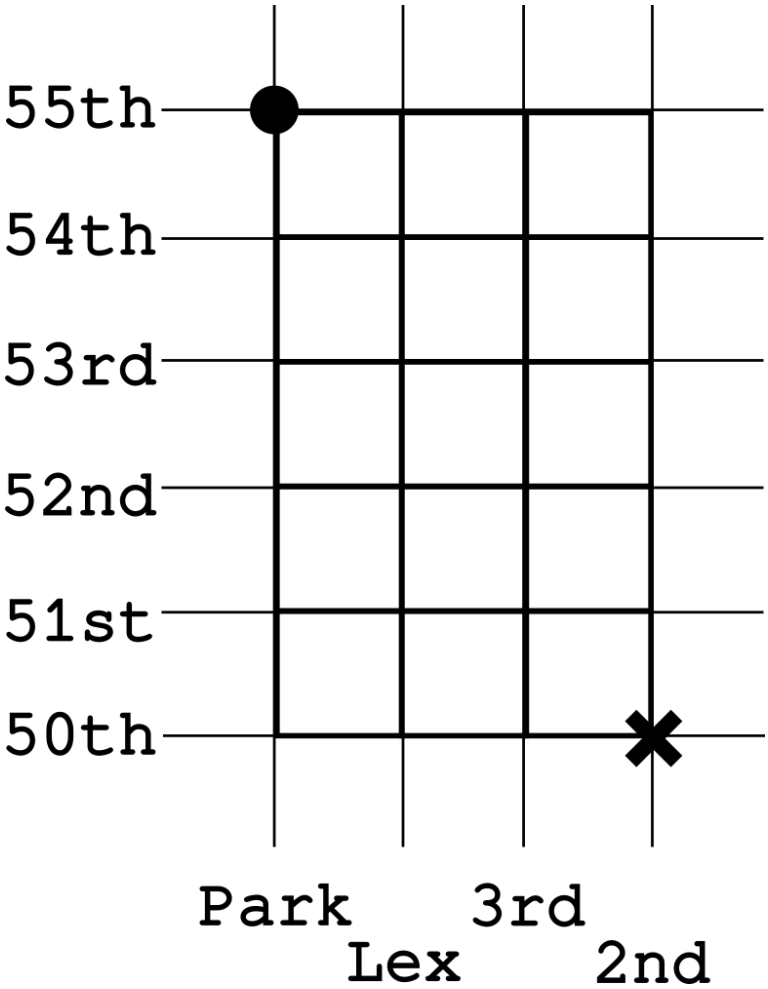
i disc = $2 \cdot 2^{i-1} - 1 + 1 - 1$ moves

i disc = $2^i - 1$ moves

In other words, the efficiency of Towers of Hanoi is $O(2^n)$ where n is the number of discs.

Manhattan Problem

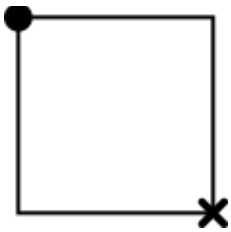
The Manhattan problem asks the question of “how many paths of shortest length” exist between two points on a grid. Distance on a grid is measured by the Manhattan distance which is constrained because travel can only occur on the grid lines themselves. While the shortest path between two points in Cartesian space is a straight line, the shortest path on the grid is the number of blocks that must be traversed in order to reach the destination.



In the above example, there are 5 blocks between 55th and 50th and there are 3 blocks between Park and 2nd and as a result the shortest path is a total of 8 blocks (5+3). However, this is only part of the problem because the actual question we are investigating is how many paths of length 8 exist between 55th and Park and 50th and 2nd. Additionally, we want to generalize our approach to be able to answer this question on an arbitrary grid, and while we could perform some brute force iterative approach, we really want to see if there is a simple and error proof way of answering this question.

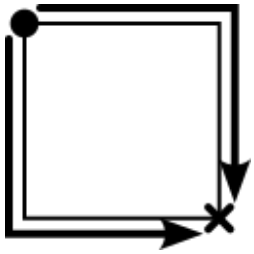
Graphical Analysis

The simplest grid is a single block:

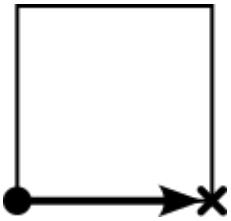
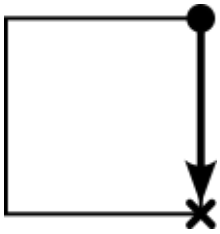
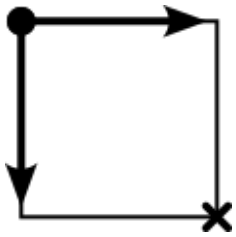


All other versions of this problem are just more extensions of this problem. If we can develop a generalized algorithm to solve this version of the problem, we can solve any version of this problem.

From a human perception and reasoning standpoint, we are inclined to quickly conclude that there are two paths from the starting position in the top-left corner to the goal in the bottom-right corner.

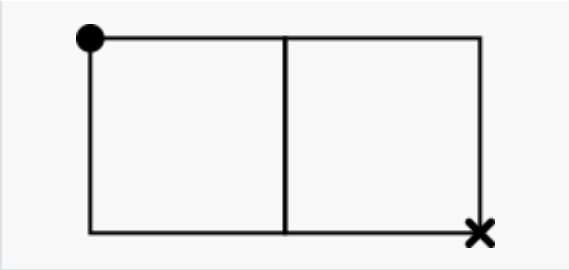


However, this overlooks how this problem can be broken down further into relevant constituents. There are really three different steps to individually consider:

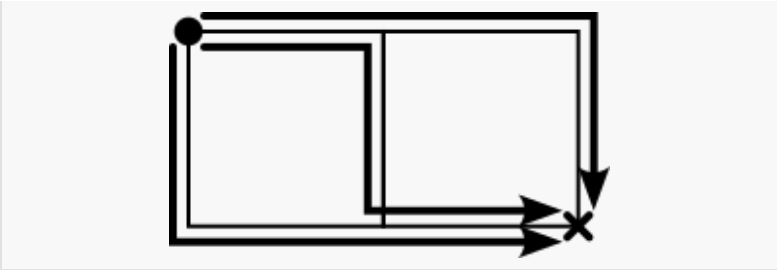
If the starting position for a path is on the bottom edge, there is only one path to the goal	**If the starting position for a path is on the right edge, there is only one path to the goal**	**If the starting position for a path is at an intersection there are multiple paths to the goal. Each path leads to a self-similar subproblem**
		
Base Case	Base Case	Recursive Case

When at an intersection, the recursive case leads to another position on the grid which may be another intersection or it may be a base case. A base case is characterized as having no choices because any direction other than moving along that edge to the goal violates shortest path, so the base cases are any position on the grid along either the bottom or right edge of this presented grid.

As argued, all more complex grids are simply extensions of the basic grid.

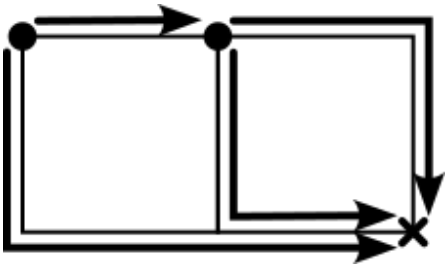


The Two Block Problem Setup

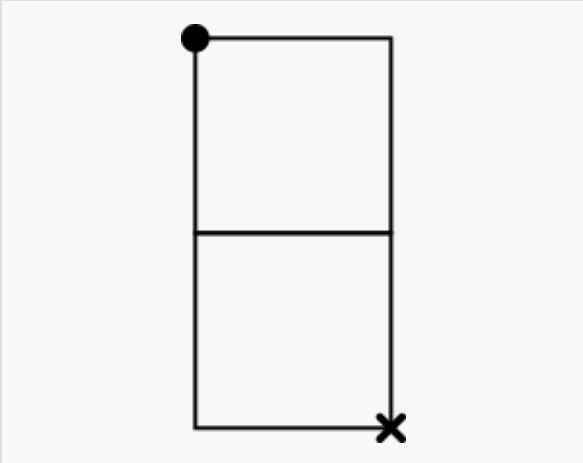


There Are Three Paths From The Start To The Goal

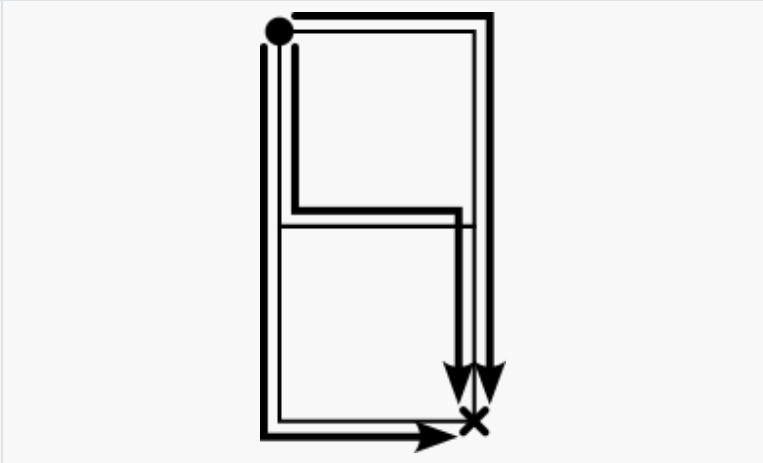
For the above example, moving down from the starting position arrives at the base case along the bottom edge which means there is only one path from the starting position when following the downward direction, and moving right from the starting position arrives at an intersection which means there is more than one path emerging from this intersection because this is self-similar, *i.e.* we are still computing the number of shortest paths from this intersection in a grid, and a sub-problem, *i.e.* we are computing a smaller grid from this intersection. If the middle intersection is reached, this is the 1 block problem discussed above.



Is there symmetry in this problem? In other words, does the above solution inform us of other similar problems? The answer is yes, as all versions of the 2x1 problem have the same number of paths..

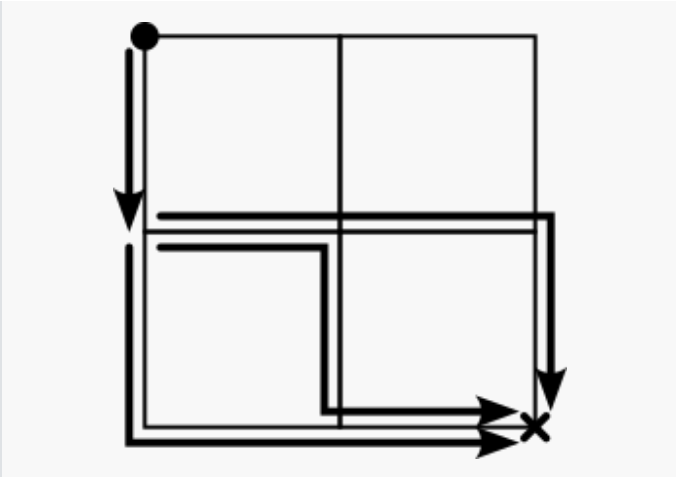
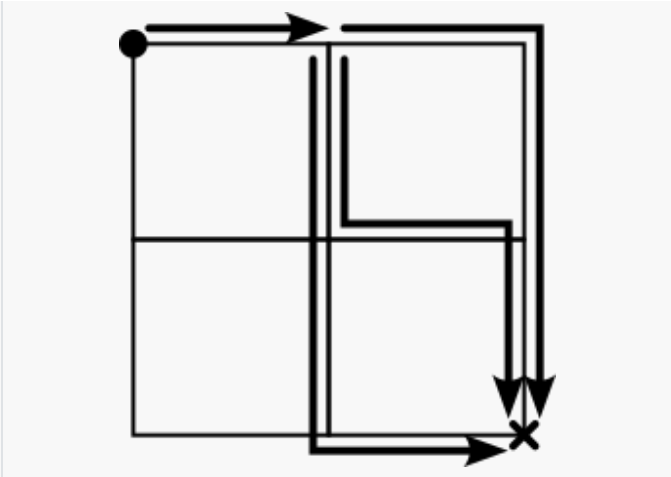


The Alternative Two Block Problem Setup



There Are Also Three Paths From The Start To The Goal

Does this symmetry further inform us? Yes, because the 2x2 problem decomposes into the 2x1 and 1x2 problems:

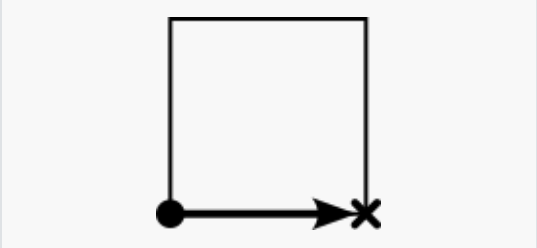
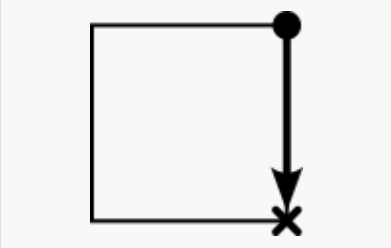
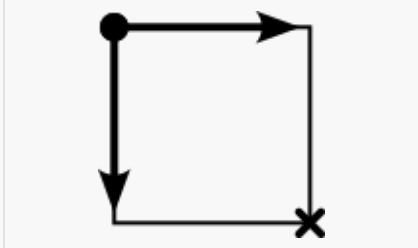
	
In The 2x2 Grid, Moving Down Leads To The 2x1 Grid Which Has 3 Paths	In The 2x2 Grid, Moving Right Leads To The 1x2 Grid Which Has 3 Paths

There are a total of 6 paths in the 2x2 grid. This problem is really the sum of the number of paths in the 2x1 grid and the number of paths in the 1x2 grid. From this observation, we see that the count of all paths can be solved by solving the adjacent subproblems.

All of the above simply reinforces that this is problem can be solved using a recursive implementation.

Recursive Implementation

Based on our analysis of the one block problem, we can identify all of the elements involved in a recursive solution to the Manhattan problem:

		
First base casetailor mage	Second base case	Recursive Case

The first base case occurs whenever we reach the bottom row of the grid regardless of which column we are in because there is only one path that may be followed of shortest distance once on the bottom row. Similarly, the second base case occurs whenever and wherever we reach the rightmost column because there is only one path of shortest distance at that point to the goal.

The recursive case occurs whenever we reach an "intersection" which is any "interior intersection" that is not along the bottommost row or rightmost column. When an intersection is reached, there always exists two directions that lead to one or more shortest paths extending from that intersection. In the basic one block problem, each of these directions lead to base cases meaning that from the two possible directions there are two and only paths total.

These observations are enough to implement a simple recursive Manhattan algorithm:

```
manhattan(row, col) → paths from this row and col :
    if row = 0  :
        return 1
    if col = 0  :
        return 1

    count = manhattan( row - 1, col )
    count += manhattan( row, col - 1 )
    return count
```

Algorithmic Time Complexity

To determine the algorithmic time complexity, revisit recursive Fibonacci. The recursive implementation of Manhattan shares similar characteristics to recursive Fibonacci. Recall that recursive Fibonacci has the following pseudocode:

```
fibonacci(x) → integer >= 0 where x is an integer and x is >= 0 :
    if x = 0 :
        return 0
    if x = 1 :
        return 1

    return fibonacci(x-1) + fibonacci(x-2)
```

So how does the above relate to the time complexity of Manhattan? From our pseudo code of Manhattan, we see the same pattern of two recursive calls to itself for each call to a recursive case. This suggests that Manhattan is at best the same efficiency as recursive Fibonacci and we determined that recursive Fibonacci is exponential or $O(2^n)$.

We could restructure Manhattan slightly and see how closely it resembles Fibonacci

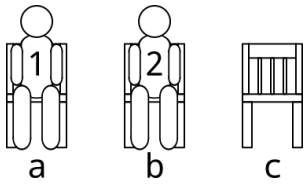
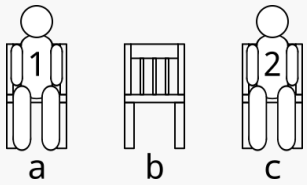
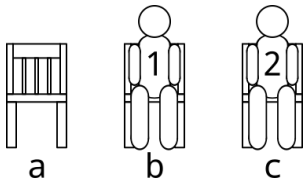
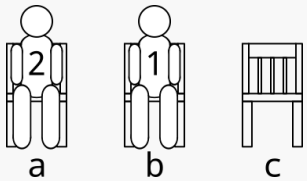
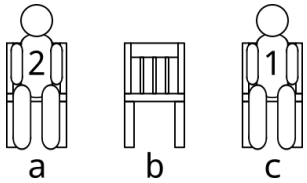
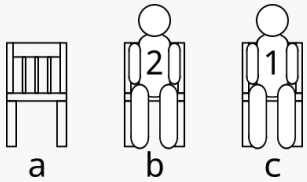
```
manhattan(row, col) → paths from this row and col :
    if row = 0  :
        return 1
    if col = 0  :
        return 1

    return manhattan( row - 1, col ) + manhattan( row, col - 1 )
```

Why did we use a more “back of the envelope” way to estimate the efficiency of Manhattan? Should we have tried to find an analytic mathematical solution that gives us a more accurate estimate? Manhattan actually belongs to a class of combinatorial problems which are similar to but have worse time complexity than exponentials, so while we classified Manhattan a little generously, the reality is that our estimate is good enough because both classes of problems are essentially unsolvable given sufficiently high input.

Permutations

A permutation problem involves making a number of choices and the order in which choices are made matters.

Imagine a stage with 3 seats (a, b, c) and two speakers (1, 2). What are all the permutations of how the seating can be arranged for the two speakers:

1	Speaker 1 could sit in seat **a** which means that speaker 2 must sit in either seat **b** or **c**. Suppose that speaker sits in seat **b**. This represents one permutation.	
2	As already suggested, speaker 2 could have instead sat in seat **c** resulting in a second permutation.	
3	Alternatively speaker 1 could sit in seat **b** which means that speaker 2 must sit in either seat **a** or **c**. Suppose that speaker 2 sits in seat **a** resulting in a third permutation.	
4	Or speaker speaker 2 sits in seat **c** resulting in a fourth permutation.	
5	Or speaker 1 sits in seat **c** which forces speaker 2 to sit in either seat **a** or **b**. If speaker 2 sits in seat a, this produces the fifth permutation	
6	If speaker 2 instead sits in seat **b**, this produces the sixth permutation	

Analysis

While the illustrative example is useful, it becomes cumbersome as the problem grows in size. Is there a mathematical formula that generalizes this idea? Where n represents the total number of elements to choose from and k represents the total choices made among the elements, the following formula predicts the number of permutations:

$P(n, k) = \frac{n!}{(n-k)!}$

Let's apply this formula to the seating problem where n is the number of seats, 3, and k is the number of speakers, 2:

$P(3, 2) = \frac{3!}{(3-2)!}$

$P(3, 2) = \frac{3!}{1!}$

$P(3, 2) = \frac{3 \cdot 2 \cdot 1}{1}$

$P(3, 2) = \frac{6}{1}$

$P(3, 2) = 6$

This formula does indeed predict the permutations. Without drawing out all the examples, what does it predict the number of permutations for 5 seats and 2 speakers to be?

$P(5, 2) = \frac{5!}{(5-2)!}$

$P(5, 2) = \frac{5!}{3!}$

$P(5, 2) = \frac{5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{3 \cdot 2 \cdot 1}$

$P(5, 2) = 5 \cdot 4$

$P(5, 2) = 20$

In short, this answer predicts that there are 20 different arrangements of speakers and chairs when there are 5 chairs and 2 speakers.

While this is a novel idea, what is the purpose of illustrating this problem? Note what mathematical operation is used in the analytical solution provided. This shows that the relationship between the size of the input n and the number of potential arrangements (the output) is a factorial relationship. In other words, permutation problems by their very nature are $O(n!)$ and are incredibly inefficient problems in terms of time complexity to solve. If the difference between n and k is sufficiently large, such as $n = 100$ and $k = 5$, then the time complexity of a permutation problem is more defined by the factorial operation. If the difference between n and k is small, such as $n = 100$ and $k = 99$, then the time complexity is closer to n.

Combinations

Permutations belong to the more general class of combinatorial problems. Combinatorial problems involve choices and the general combination problem does not consider the order of choices while permutation problems do. For example choosing six for one die role does not limit the choices for another die roll (a combination problem) but choosing six for this die role does limit the remaining choices for this die role assuming exclusivity (a permutation problem). The general equation for a combinatorial is also factorial and is defined by the equation:

$C(n, k) = \frac{n!}{k!(n-k)!}$

Unfortunately, most of the really interesting problems in modern society are choice problems and so are defined by combinations or permutations.

Manhattan above is actually a combinatoric problem even though our rough estimation predicts it to be exponential. Calculating the number of paths using an analytical equation is difficult for those who have not focused on combinatorics, but it is quite easy to implement an algorithm to solve the equation; regardless of our mathematical background, as programmers we still need to practical ways to address the question of how much time will the algorithm take to solve the problem.

Exploration Problems

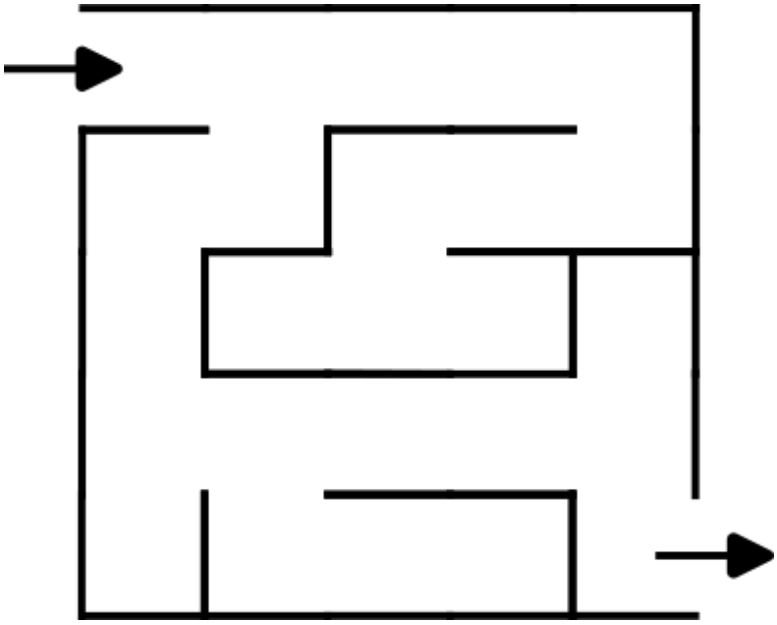
An exploration problem is one that cannot be solved without some flexibility to undo a decision. For example, if we were not able to undo a decision when exploring a maze, we would quickly become trapped in the maze. It is the ability to retrace our steps that allows us to escape a maze whether it is solvable or not.

A recursive algorithm that has a well defined solution and does NOT require exploration is called **tail recursive**. It is called this because these algorithms typically end with a recursive call at the tail-end of their definitions.

We call the algorithmic strategy of retracing steps and undoing decisions **backtracking**. Backtracking can be applied to a variety of problems, but in general, *we need the ability to backtrack in problems where we do not know whether there is a solution or not*. These such problems need to be robust to failure as they may have many different failure states and few if any paths to success.

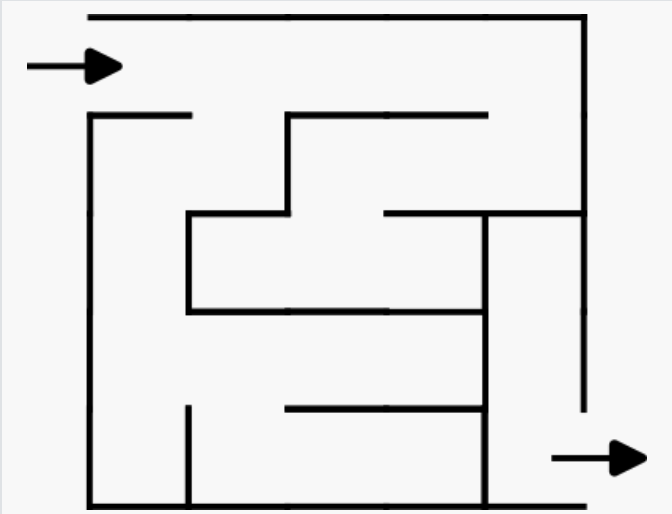
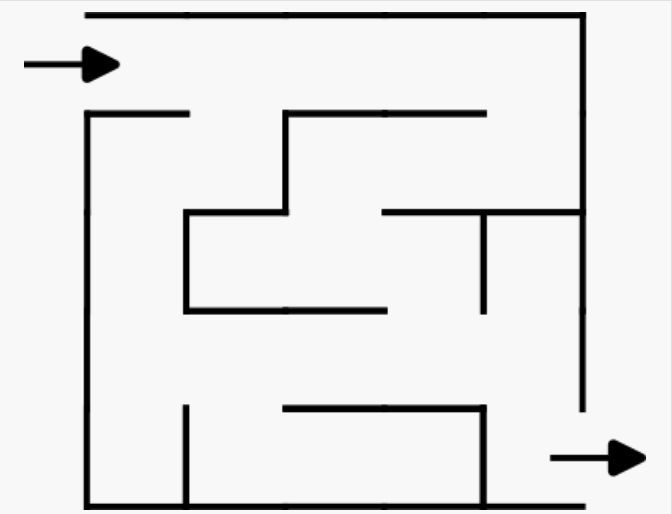
Backtracking algorithms are often more easily expressed through recursive implementations because recursion is naturally built upon the memory Stack. The primary challenge to writing a backtracking algorithm is to manage state restoration when it is necessary to undo a decision. If using a loop and a table (multi-dimensional array), it can be difficult to accurately compute earlier state efficiently. Because recursion stores the history of previous unresolved function calls and associated parameters on the memory Stack, there is generally a natural snapshot of earlier state readily available in memory and restoration is automatically handled by the system. One key feature of the stack data structure, which we will study in depth in Chapter 9, is that it is ideally suited for maintaining a history of information, so because we are using a stack to manage memory, there is a mechanism built into Java memory management tailor made for backtracking through recursion which is why a recursive algorithm is a natural fit for these types of problems and the recursive implementations generally result in deceptively simple implementations, simple because they don't have extensive and explicit state management in the implementation.

Maze



First consider that solving the maze from inside limits the information available and the top down map we have here is not generally information we have when exploring. Instead, we have to put ourselves inside the maze to have the proper perspective about our inherent limitations.

To explore a maze successfully we need to be flexible but robust. There are mazes with no solution and there are mazes where we can wander in circles and that may have multiple solutions. For this problem we are only concerned with finding a single solution and not all solutions.

	
A maze with no solution	A maze with multiple solutions and an infinite cycle

We will assume we are dealing with a maze composed of square cells oriented to a grid describable by the cardinal directions. Each cell can be entered or exited along a direction of travel if there is no wall in that direction. We call a cell with *more* than two directions of travel an "intersection", a cell with only two directions of travel a "hallway", and a cell with only one direction of travel a "dead-end".

A maze has a starting point called an "entrance" and a goal called an "exit" and to traverse a maze is to find a path consisting of one or more cells with contiguous directions of travel that link start to goal.

At intersections there are multiple choices about the directions one may go and represent points in the maze where a decision must be made. However, only on direction can be explored at a time, so it may be necessary to return to an intersection later if exploration of another direction of travel fails.

Hallways do not represent choices. A hallway connects to an intersection, a dead end, the goal, or the starting point.

Dead-ends represent local failures in an exploration but reaching a dead end does not suggest the problem is unsolvable, it simply suggests that the sequence of choices made results in a local failure but it says nothing about other sequences of choices.

If the goal is ever reached, a path through the maze has been found and traversal has succeeded. If the start is ever arrived at again after traversal is started, assuming all decisions have been explored, then the maze has no solution and cannot be traversed.

To consistently navigate and avoid retracing already made decisions, potential paths at intersections should be explored in a clockwise order and breadcrumbs can be used to indicate that a location has already been visited.

Due to the extreme variations among mazes and problems like circularity, it might be very difficult to successfully write an iterative algorithm that solves the maze. Can we use a recursive strategy instead?

Consider that at each intersection, each path ahead is simply a smaller maze, so a maze is composed of self-similar subproblems which means that intersections represent recursive cases. Base cases occur whenever a dead-end, goal, or starting point is visited.

A dead-end is just a local failure, so the response is to backtrack to the most recent intersection (decision) and explore the next unexplored path. If all paths at this intersection have been explored, the response is to backtrack further to earlier intersections until an unexplored direction of travel is found. If backtracking leads all the way back to the starting point, there is no solution because all decisions have been explored. If the goal is reached instead of a dead-end, there exists a path from start to goal and this path is recorded in the current list of successful decisions used to get to the goal.

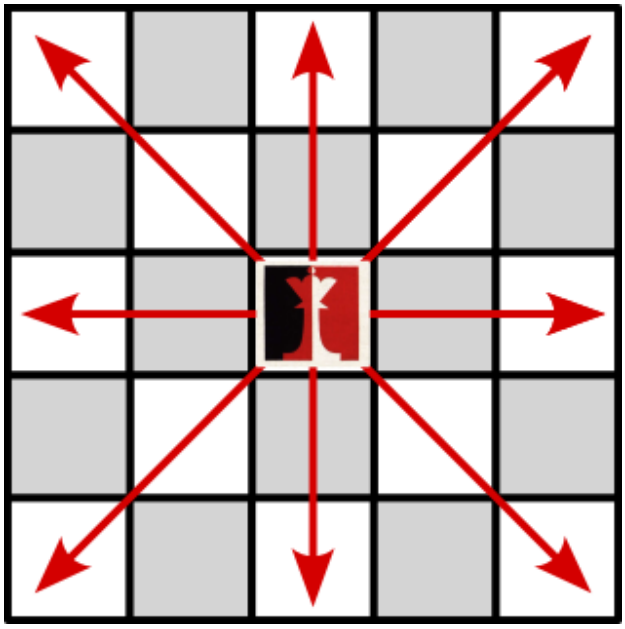
Backtracking

Backtracking is the power to undo decisions and typically involves a stack to retain the history of decisions. This history is naturally stored in reverse order as the bottom of the stack represents the first decision and the top of the stack represents the most recent decision. The memory Stack is a stack so it naturally supports backtracking and since the Stack manages memory for recursion anyway, *i.e.* the stack records state for each function call and all local variables are stored on the stack, backtracking is generally accessible and very natural for recursive implementations. We will talk more about stack data structures in a few weeks.

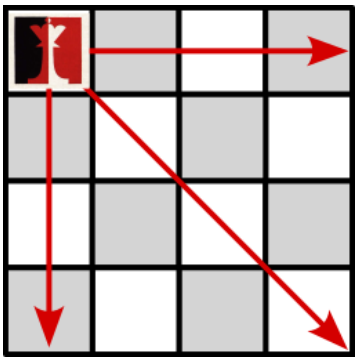
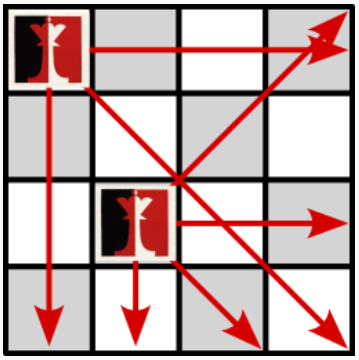
Backtracking is a useful strategy for solving “exploration problems”. An exploration problem is one where we are not sure if there is a solution to begin with and to determine if there is a solution it requires an exploration of the solution space. Examples of exploration problems are a maze, hill climbing problems like evolutionary problems, and n-queens.

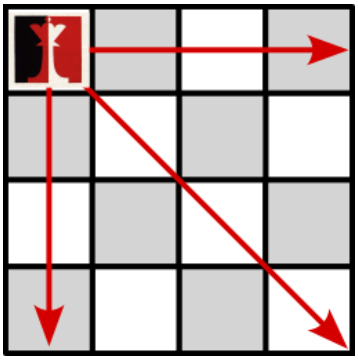
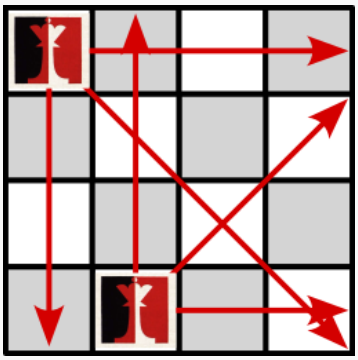
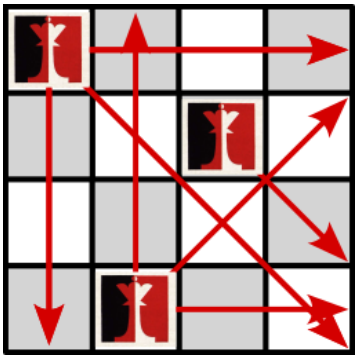
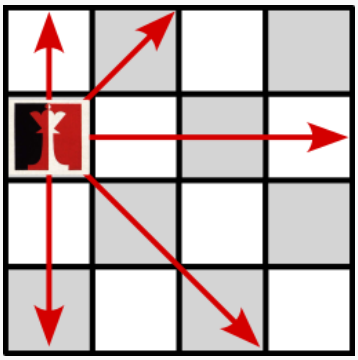
N-Queens

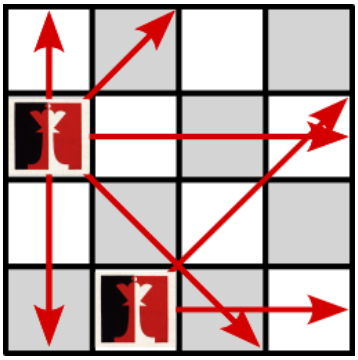
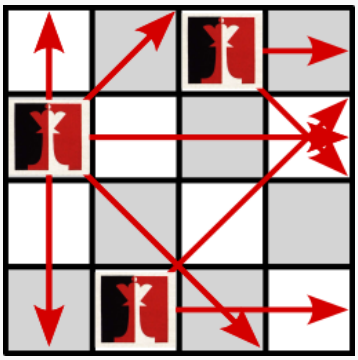
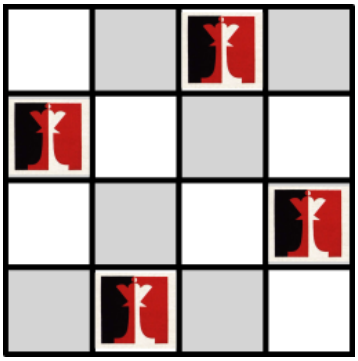
In chess, the queen has complete movement and is the most powerful piece on the chess board. It can attack in any direction, vertical, horizontal, and along either diagonal, so any piece that ends its movement along one of these axes is threatened by the queen and may be captured on the queen's next movement.



- We saw that N-queens is a backtracking problem as well
- We may call it "trial and error", but we used a very structured strategy

has1. Place a queen in the first column, first row	
2. Try to place a queen in the second column. We first try to place the queen in the first row but it is threatened by the queen in the first column. We then try to place the queen in the second row but it is also threatened by the queen in the first column. We then try to place the queen in the second column, third row and succeed because this position is not threatened by preexisting queens on the board.	

3. We cannot place a queen in the third column because all positions in the column are threatened by the placement of queens in the first and second columns. This represents a failure in the process, but it does not mean it is unsolvable. Our next step is to remove the most recent queen's placement (placed in the second column, third row) restoring the previous board state, and try to place it elsewhere if possible.	
4. Since the previous attempt was the second column, third row, we continue on to the fourth row and try that position. The second column, fourth row is unthreatened, so we replace the queen there and continue the process.	
5. Attempt to place in the third column first by testing the third column first row which we find under threat and not a legal position. We then find the third column, second row to be not under threat, so we place the queen there. However, scanning the fourth column shows that all positions are under threat, so placement in the third column, second row does not lead to a solution. Subsequent placements in the third column are all illegal, so there is no solution for placing the queen in the second column, fourth position. Because this is the last row in the second column, it means that our initial placement of the queen in the first column, first row has no solutions, and we must replace the queen in the first column.	
6. We have undone each column and replaced the first queen in the first column, second row.	

7. We try all positions in the second column beginning with the first row and find positions up to the last position is under threat from the queen in the first column, second row. Because the last positon is legal, we place the queen in the second, column, fourth row.	
8. We start at the top of the third column and scan top to bottom for a legal position to place the queen. We find that the third column, first row is unthreatened, so we place a queen there and continue on to try to place in the fourth column.	
9. We start at the top of the fourth column and scan top to bottom for a legal position to place the last queen. We find that the fourth column, third row is unthreatened, so we place a queen there. Because we have placed our last queen, we have found a solution and the process is both successful and complete.	

Recursive Implementation

```
nqueens(n, c) → n = the number of queens to assign, c = the column to start with : boolean
indicating if solved
    if n == 0 :
        return true
    if c == 0 :
        return false

    for each row r :
        if [r,c] is not threatened(r,c) :
            place queen n at [r,c]
            found = nqueens( n-1, c-1 )
            if found :
                return true
    return false
```

Algorithmic Time Complexity

The obvious aspect is that for each recursive call, the algorithm proceeds through each row and attempts to place a queen; however, the time complexity is seriously affected by the efficiency of more opaque aspects of the algorithm.

One opaque aspect is that there is some magic operation `threatened(r,c)` which returns a boolean that summarizes whether the cell in question is legal for a queen's placement. While a lookup of this information in a table could be as fast as $O(c)$ the efficiency of the update whenever a queen is placed or removed is unclear and could be as much as $O(n^2)$ itself.

Regardless of the efficiency of `threatened` the overall efficiency of the placement and removal process is not straightforward. It is not simply a loop inside a loop as that would only be possible if we never removed a queen; however, this is a robust process intended to backtrack whenever a queen is placed in an unsolvable position. Each call to `nqueens` represents iterating over every row in a given column and it is possible to backtrack from every previous column configuration when a failure is detected. As a result, in the worst case, `nqueens` has the potential to try every permutation of queen placement on the board plus the additional cost of updating the `threatened` data structure on attempting to place a queen and removing a queen.

Because `nqueens` is a form of permutation problem, then this must be operating in on the order of factorial time and can be estimated with a time complexity of $O(n!)$ where n is the size of the input equal to both the number of queens and the number of rows and columns on the board. While for 4 queens this still seems like a trivial problem, consider how complex it would be to solve this for $n = 100$ or $n = 1000$.

Review

1. What is recursion?
2. Is the piecewise definition of power recursive, and if so, why?
3. Is it possible to do express everything recursively?

4. What are the behaviors and characteristics of an iterative design?
5. What are the behaviors and characteristics of an recursive design?
6. What types of problems are good candidates for recursive implementations? Why?
7. What is meant by "a problem that can be reduced into self-similar subproblems"?
8. Why are reductive problems good candidates?
9. What are the two cases necessary to implement a recursive design and what are their characteristics?
10. How do we structure recursive implementations with regard to these cases? Why?
11. How do we control recursion? *Note: A base case alone does nothing, so it alone cannot control recursion.*
12. Why do we try to model base cases at or around zero ,one, or null cases?
13. Analyze the time complexity of iterative power.
14. Analyze the time complexity of recursive power.
15. Analyze the time complexity of iterative factorial.
16. Analyze the time complexity of recursive factorual.
17. Analyze the time complexity of iterative fibonacci.
18. Analyze the time complexity of recursive fibonacci.
19. Why is the stack so important to recursive algorithms? What is it handling for us?
20. What happens if recursion is uncontrolled?
21. Given the previous question, why is there a finite number of recursive calls that can be made and why?
22. What is the relationship between the number of variables (and parameters) in a recursive function and the upper limit on the number of recursive calls?
23. Give an example of a permutation
24. Formally define permutations in the context of available options, numbers of choices, and order of choices. *An illustrative example would be enough.*
25. If permutations are modeled by the formula $P(n,k) = \frac{n!}{(n-k)!}$ and combinations are modeled by the formula $C(n,k) = \frac{n!}{(n-k)!k!}$, what is the general algorithmic efficiency in terms of Big O of any algorithm that attempts to solve a permutation or combination problem by brute force? Why?
26. What is the "Manhattan Distance"?
27. Define the "Manhattan Problem" in terms of Manhattan Distance.
28. What are the constraints on the Manhattan Problem?
29. Illustrate the base cases for the Manhattan Problem.
30. Illustrate the recursive case for the Manhattan Problem.
31. Write pseudo code that solves the Manhattan Problem.
32. Estimate the growth of the problem with respect to the number of operations for the Manhattan Problem from the pseudo code.
33. What does it mean to backtrack?

34. Why is recursion a natural approach for backtracking problems?
35. Why might an iterative approach be difficult to implement for a backtracking problem?
36. What is an “exploration problem”?
37. Why are exploration problems good candidates for backtracking problems
38. Describe the Maze problem.
39. What is the “self-similar subproblem” in a maze?
40. Why is a Maze an exploration problem?
41. Describe backtracking in a Maze.
42. Describe the N-Queens problem.
43. What is the “self-similar subproblem” in N-Queens?
44. Why is the N-Queens problem an exploration problem?
45. Describe backtracking in the N-Queens example.
46. Analyze the time complexity of N-Queens
47. Describe the Towers of Hanoi problem.
48. Does Towers of Hanoi involve backtracking? Justify your answer.
49. What is an application of Towers of Hanoi? Justify your answer.
50. Analyze the time complexity of Towers of Hanoi
51. Contrast “tail recursion” with “backtracking” recursion.
52. Why is it more “expensive” and more prone to error to count up to a target value for a base case rather than to center that base case around one, zero, or null? What is it more expensive in terms of and why does this expense further limit recursion?
53. If iterative approaches are almost always less expensive in terms of memory than recursive approaches, why use recursive approaches at all?